



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI**
Department of Artificial Intelligence & Data Science
CS23633 – Cloud Computing Laboratory

Experiment 2

PAAS in Public Cloud Using Render

AIM

To build a basic Python Flask web application and deploy it on the Render cloud platform. This experiment helps in understanding how a Platform as a Service (PaaS) allows applications to run online without manually setting up or maintaining servers.

What is PaaS?

Platform as a Service (PaaS) is a cloud service that provides everything needed to develop, deploy, and run an application. It takes care of server management, software installation, and deployment, allowing developers to focus only on writing the application.

Why Render?

Render is an easy-to-use cloud hosting platform that supports web application deployment directly from GitHub. It automatically builds and hosts the application, making deployment simple even for beginners.

PROCEDURE:

Step 1

Sign in to your GitHub account or create one if you do not already have an account.

Step 2

Create a project folder named **RenderHelloApp**.

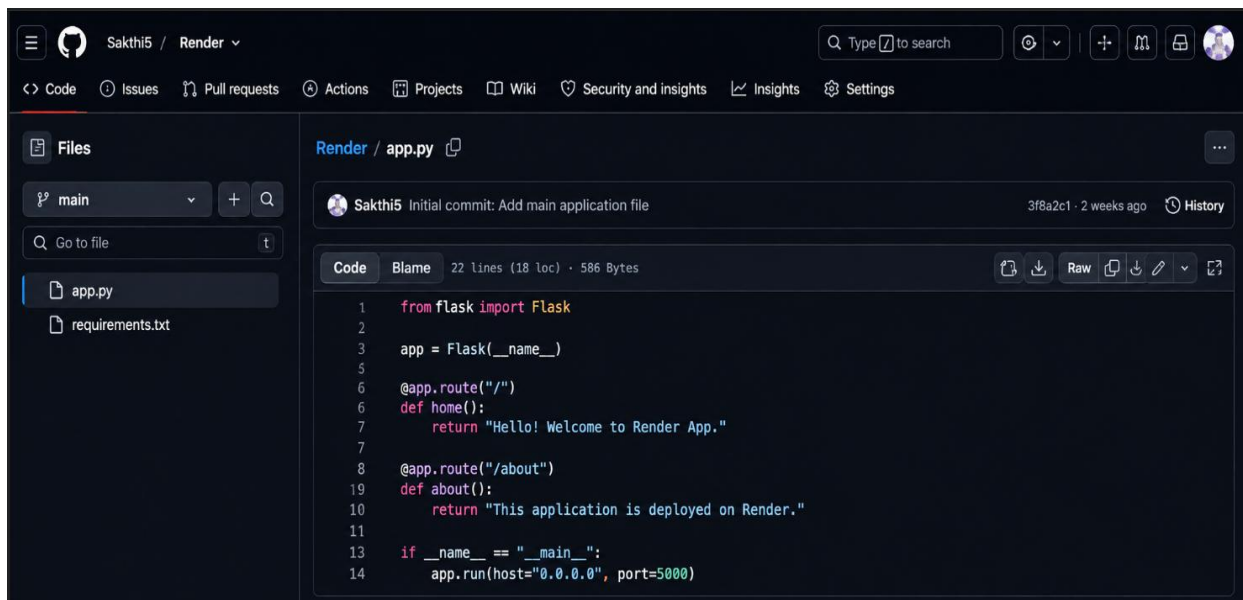
Step 3

Inside the project folder, create the following files:

- app.py
- requirements.txt

Step 4

Write the Flask application code in the app.py file.

A screenshot of a GitHub repository interface. The repository is named 'Render' and is owned by 'Sakthi5'. The 'app.py' file is selected in the file explorer on the left. The main content area shows the code for 'app.py', which is a Flask application. The code includes imports for Flask, defines a Flask app, and sets up two routes: a home route that returns 'Hello! Welcome to Render App.' and an about route that returns 'This application is deployed on Render.'. The code also includes a main block that runs the app on host '0.0.0.0' and port '5000'. The commit history shows an initial commit by 'Sakthi5' titled 'Add main application file' from 2 weeks ago.

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5
6 @app.route("/")
6 def home():
7     return "Hello! Welcome to Render App."
7
8 @app.route("/about")
19 def about():
10     return "This application is deployed on Render."
11
13 if __name__ == "__main__":
14     app.run(host="0.0.0.0", port=5000)
```

Step 5

Add the required Python packages such as **Flask** and **Gunicorn** to the requirements.txt file.

Step 6

Create a new public repository called **RenderHelloApp** on GitHub.

Step 7

Upload both project files to the repository and commit the changes.

Step 8

Create an account on Render and connect it to your GitHub account.

Step 9

Create a new Web Service in Render.

Create a new **Key Value Instance**

1 Choose service > 2 Configure > 3 Deploy

- Static Site
- Web Service
- Private Service
- Workflow **Beta**
- Background Worker
- Cron Job
- Postgres
- Key Value
- Project
- Blueprint

Static Sites
Static content served over a global CDN. Ideal for fronted, blogs, and content sites.
[New Static Site →](#)

Web Services
Dynamic web apps ideal for full-stack apps, API servers, and mobile backends.
[New Web Service →](#)

Private Services
Web apps hosted on a private network accessible only from your other Render services.
[New Private Service →](#)

Cron Jobs
Short-lived tasks that run on a periodic schedule.
[New Cron Job →](#)

Postgres
Relational data storage. Supports point-in-time recovery, read replicas, and high availability.
[New Postgres →](#)

Key Value
Managed Redis®-compatible storage. Ideal for use as a shared cache, message broker, or job queue.
[New Key Value Instance →](#)

Workflow **Beta**
Run thousands of parallel tasks with zero ops overhead.
[New Workflow →](#)

Workers
Process async queues.
[New Worker →](#)

Select the **RenderHelloApp** repository from GitHub.

Step 11

Enter the deployment details:

- **Language:** Python
- **Branch:** main
- **Root Directory:** Leave empty
- **Build Command:** pip install -r requirements.txt
- **Start Command:** gunicorn app:app
- **Instance Type:** Free (if available)

Root Directory Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Build Command

Render runs this command to build your app before each deploy.

Start Command

Render runs the command to start your app with each deploy.

Instance Type**For hobby projects****Free**

\$0 / month

512 MB RAM

0.1 CPU

**Upgrade to enable more features**

Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs or persistent disks. Select a paid instance type to enable these features.

For professional use

For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support:

- Zero Downtime

**Starter**

\$7 / month

512 MB RAM

0.5 CPU

**Standard**

\$25 / month

2 GB RAM

1 CPU

**Pro**

\$85 / month

4 GB RAM

2 CPU

**Pro Plus**

\$175 / month

8 GB RAM

4 CPU

Step 12

Click Deploy Web Service and wait for the deployment to complete.

Environment Variables

Set environment-specific config and secrets (such as API keys). Then read those values from your code. [Learn more.](#)

Generate



Add Environment Variable

Add from .env

Advanced

Deploy Web Service

Step 13

After deployment, open the URL generated by Render and check whether the home page is displayed correctly.

231801505

Step 14

Add **/about** at the end of the application URL to verify that the second route is also working properly.

The screenshot displays the Render Sakthi dashboard. The left sidebar contains navigation links: Web Services, Render Sakthi, Events, Settings, MONITOR (Logs, Metrics), and MANAGE (Environment, Shell, Scaling, Previews, Jobs, Changelog, Invite a friend, Contact support). The main content area shows the 'Render Sakthi' service details, including the Service ID, repository link, and a 'Manual Deploy' button. A warning message indicates that the free instance will spin down with inactivity. Below this, the deployment status is shown as 'Live' with a timestamp of 'July 30, 2026 at 9:56 PM'. The logs section shows the following output:

```
Jul 30 21:56:01 ==>Using cached Flask-3.0.3-py3-none-any.whl.metadata (3.6 kB)
Jul 30 21:56:01 Collecting gunicorn==21.2.0 (from -r requirements.txt (line 1))
Jul 30 21:56:01 Using cached gunicorn-21.2.0-py3-none-any.whl.metadata (4.4 kB)
Jul 30 21:56:01 Collecting Werkzeug==3.0.4 (from Flask==3.0.3->-r requirements.txt (line 1))
Jul 30 21:56:01 Using cached werkzeug-3.0.4-py3-none-any.whl.metadata (4.8 kB)
```

The browser address bar shows the URL `rendersakthi.onrender.com` and the page content displays 'Hello! Welcome to Render Sakthi.'

CONCLUSION:

This experiment demonstrated how a Flask application can be deployed using Render, a Platform as a Service. I learned that PaaS simplifies application deployment by handling the server setup and hosting, allowing developers to publish applications quickly without managing infrastructure.